Prolog

- Prolog (Programming in Logic) is a declarative programming language based on first order logic.
- Instead of telling the computer *how* to do something, in Prolog, you declare facts, rules, and relationships, and let the system deduce answers.
- Widely used in Artificial Intelligence, especially for knowledge representation, expert systems, natural language processing**,** and reasoning.
- Uses backward chaining and unification (pattern matching) to answer queries: it starts from the goal and tries to match facts or rules recursively.

Predicates

- A predicate defines a relation between arguments (facts or rules).

- Syntax:
```
predicate_name(arg1, arg2, ..., argN).
```

- Examples:
```
weather(karachi, hot).        % fact
parent(alice, bob).           % fact
comfortable(X) :- weather(X, moderate).  % rule
```

- Predicate names:
  - must start with a lowercase letter.
  - can include letters, digits, and underscores (_).
- Every statement ends with a period (**.**).

Arguments

- Arguments can be atoms or numbers or variables.
- Atoms and numbers are constants here.

- Atoms (names/symbols, are lowercase)
- Examples:
```
city(karachi).
city('New York').
```

- Numbers (integer or float)
- Examples:
```
age(ali, 25).
height(alice, 1.7).
```

- Variables (unknowns, start with uppercase or _)
- Example:
```
comfortable(X) :- weather(X, moderate).
```

- Constants (atoms, numbers) do not change. Variables are placeholders.

Facts

- State something specific that is true.

- Example:
```
parent(alice, bob).        %fact
```
- Facts usually do not use variables.

Rules

- State general truths using variables.
- Allow generalization over multiple facts

- Syntax:
  `head :- body.`
  - `head` is the predicate being defined.
  - `body` contains subgoals.
  - `:-` is the rule separator.
  - To combine more than one subgoal `,` for AND and for `;` OR can be used.

- Examples:

```
comfortable(X) :- weather(X, moderate).
                % X is comfortable if X's weather is moderate
chilly(X) :- weather(X, cold), season(X, winter).
hot_area(X) :- weather(X, hot); season(X, summer).
```

Queries

- Ask Prolog if a goal can be satisfied.
- Start with `?-`.

- Example:

```
?- comfortable(X).
```

- Variables in queries are placeholders Prolog will try to fill.
- Demonstrate multiple answers and using `;` to see all possibilities.

Built-in Predicates for Output Formatting

- `write(Text)/1` → prints text.
- `nl/0` → new line.
- Example:

```
write('Hello'), nl.
```

Case Sensitivity

- Predicate names and atoms → lowercase.
- Variables → uppercase or _.

Backward Chaining and Backtracking

1. Fills variables via unification (pattern matching)
2. Backtracks to find all solutions

- Example
  Knowledge Base

```
% === Facts ===
weather(karachi, hot).
weather(lahore, moderate).
weather(peshawar, moderate).
weather(islamabad, cold).

% === Rules ===
comfortable(X) :- weather(X, moderate).
chilly(X) :- weather(X, cold).
```

Backtracking with a Factual Query

- Querying all cities with moderate weather directly from facts:
  `?- weather(X, moderate).`

- Prolog Execution:
  1. Looks at the first fact: `weather(karachi, hot)` → doesn't match
  2. Looks at second fact: `weather(lahore, moderate)` → matches → `X = lahore`
  3. You press `;` → Prolog backtracks to find another match
  4. Looks at third fact: `weather(peshawar, moderate)` → matches → `X = peshawar`
  5. Looks at fourth fact: `weather(islamabad, cold)` → doesn't match → no more matches
  6. Returns `false` if you press `;` again
- Output:

```
X = lahore ;
X = peshawar ;
false.
```

Backtracking with a Rule Query

- Querying cities that are comfortable:
  `?- comfortable(X).`

- Prolog Execution (Backward Chaining):
  1. Goal: `comfortable(X)`
  2. Matches rule: `comfortable(X) :- weather(X, moderate).`
  3. Subgoal: `weather(X, moderate)`
  4. Finds `X = lahore` → first solution
  5. You press `;` → backtracks to find next match
  6. Finds `X = peshawar` → second solution
  7. No more matches → Prolog ends → `false` if you press `;` again

Output:

```
X = lahore ;
X = peshawar ;
false.
```

A Quick Cheat Sheet

Syntax / Construct	Description	Example
`%`	Comments	`% this is a comment`
`. (period)`	Terminates a fact, rule, or query	`parent(alice, bob).`
`?-`	Prolog shell prompt for queries	`?- likes(bob, tea).`
`Variables`	Start with uppercase letters or _	`X, Person, _Temp`
`Constants / Atoms`	Start with lowercase or quoted string	`alice, 'New York'`
`Fact`	Unconditionally true statement	`likes(alice, chocolate).`
`Rule`	Conditional statement	`happy(X) :- likes(X, chocolate), has_money(X).`
`,`	Logical AND	`likes(X, tea), likes(X, coffee).`
`;`	Logical OR	`likes(X, tea); likes(X, coffee).`
`\+`	Negation / not provable	`\+ likes(bob, coffee).`
`_`	Anonymous variable / wildcard	`parent(_, bob).` `% anyone who is parent of Bob`
`%`	Single-line comment	`% This is a comment`
`/* ... */`	Multi-line comment	`/* Multi-line comment */`

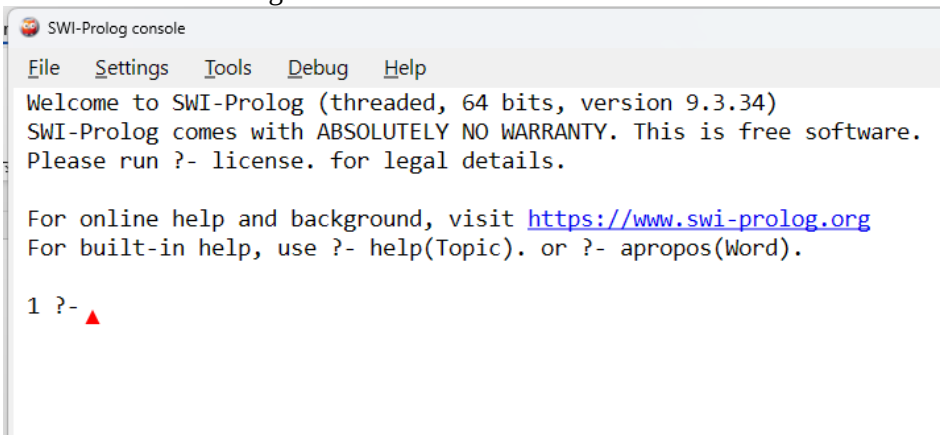
Installation

- Download and install SWI-Prolog from:
  https://www.swi-prolog.org/

How to Run Programs using SWI-Prolog

1. Write your program
   Open any text editor, write your code, and save the file with the .pl extension.

2. Open SWI-Prolog
   Launch the SWI-Prolog environment.



3. Load your program
   You can load the program in one of two ways:
   - From the menu: File → Consult..., then select your .pl file.
   - Or from the command line:
     `?- consult('<PATH>/Prolog/weather.pl').`
     If the file loads successfully, Prolog will display:
     `true.`

4. Run your program
   Execute the main goal using:
   `?- start.`

5. Reload after editing
   If you modify your program, it can be reloaded in one of the two ways:
   - From the menu: File → Reload modified files
   - Or from command line:
     `?- make.`

6. Exit Prolog
   To exit the interpreter:
   `?- halt.`
   Press Enter.
   You can restart Prolog later to load another program (useful if different programs share variable names or conflicting rules).

Exercises

Attach code and snapshot of the output for the following exercises in your workbook in lab 10.

1. Create facts about city populations:
   - Population of Karachi is 15000000.
   - Population of Lahore is 11000000.
   - Population of Islamabad is 2000000.
   - Population of Quetta is 1000000.
   - Population of Peshawar is 2500000.

   Define the following rules:
   - A city is large if population > 5,000,000.
     [Hint: `large_city(X) :- population(X, P), P > 5000000.`]
   - A city is small if population < 3,000,000.

   Run the following queries:
   - Find all large cities.
   - Find all small cities
   - Get population of Lahore.
   - Get all the cities where population is greater than 2000000. [Hint: create a new rule for this.]

2. Use your existing weather and season facts to create a weather and travel advisory.

   Define rules for travel advisories:
   - A city is good_for_travel if it is comfortable and not hot
     [Hint: `good_for_travel(X) :- comfortable(X), \+ hot_area(X).`]
   - Avoid_travel if it is chilly or hot_area
   - Moderate_advice if neither good nor avoid

   Run the following queries:
   - List cities suitable for travel
   - List cities to avoid
   - Cities with moderate advice